НИТУ «МИСИС»

Кафедра инженерной кибернетики

ОТЧЕТ

ПО

ЛАБОРАТОРНОЙ РАБОТЕ №8.1

«Разработка программ с поддержкой технологии CUDA

с использованием компилятора командной строки»

учебная дисциплина «Параллельные вычисления»

Студент: Голубев И. М., БПМ-22-ПО-1

Преподаватель: Ватутин Э. И.

Оценка:

Дата защиты: 27.04.2026

2026 г.

Введение

Основная цель работы:

Познакомиться с особенностями разработки программ с использованием технологии CUDA.

Описание решаемой задачи

Разработать следующую программу:

a. Разработать CUDA-ядро для поэлементного сложения векторов;

b. Сформировать вектора из случайных исходных данных;

c. Найти сумму векторов с использованием разработанного CUDA-ядра и с использованием подпрограммы на CPU, сравнить полученные результаты, сделать вывод о корректности расчетов на GPU.

Входные данные

Два массива длины 1024 со значениями, случайно равномерно распределёнными от -100 до +100.

Результат выполнения программы

В ходе выполнения программы должны быть заполнены два массива:

1. Массив, вычисленный на CPU: c_{cpu}

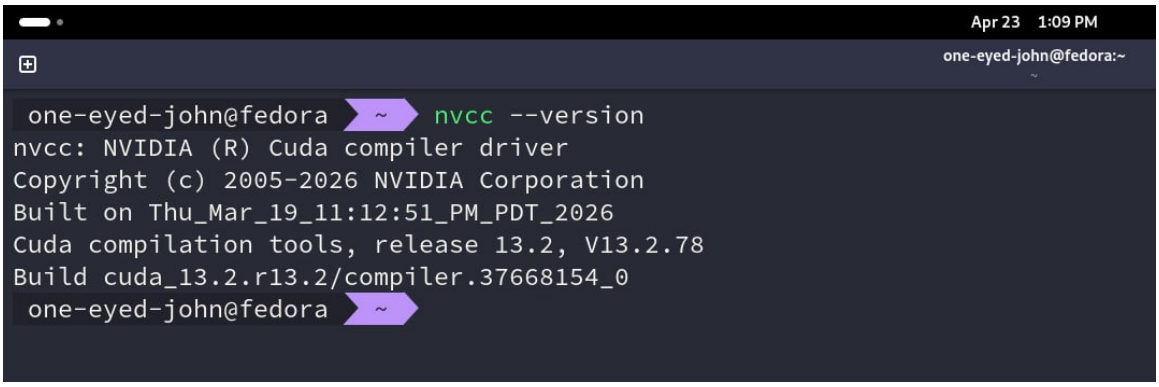
2. Массив, вычисленный на GPU: c_{gpu}

Далее необходимо вычислить ошибку между значениями двух полученных массивов c_{cpu} и c_{gpu}, а затем вывести пользователю значение максимальной ошибки и первые 20 элементов массива, посчитанного на GPU.

Выбранные средства для разработки

Для выполнения лабораторной работы использовались следующие программные средства и технологии:

- Язык программирования: C++17
- Среда разработки: Zed
- Операционная система: Fedora Linux
- Процессор: Intel(R) Core(TM) i5-10300H CPU @ 2.50GHz
- Размер RAM: 15Gi
- Компиляция выполняется средствами утилиты `nvcc`



Листинг кода программы

Код доступен в удалённом Github репозитории по ссылке:

https://github.com/g0lubew-iv/parallel-comp/blob/main/lab08/lab08_1.cpp

https://github.com/g0lubew-iv/parallel-comp/blob/main/lab08/lab08_1.cu

Файл lab08_1.cpp

```
#include <iostream>
#include <random>
#include <cmath>

using namespace std;

extern "C" void vec_add_cuda(const float *a, const float *b, float *c, int n);

static void vec_add_cpu(const float *a, const float *b, float *c, int n) {
    for (int i = 0; i < n; i++) {
        c[i] = a[i] + b[i];
    }
}

int main() {
    const int N = 1024;

    float a[N], b[N], c_gpu[N], c_cpu[N];

    std::mt19937 rng(123);
    std::uniform_real_distribution<float> dist(-100.0f, 100.0f);

    for (int i = 0; i < N; i++) {
        a[i] = dist(rng);
```

```
        b[i] = dist(rng);
        c_gpu[i] = 0.0f;
        c_cpu[i] = 0.0f;
    }

    // GPU
    vec_add_cuda(a, b, c_gpu, N);

    // CPU
    vec_add_cpu(a, b, c_cpu, N);

    // compare
    float maxAbsErr = 0.0f;
    int worstIdx = 0;

    for (int i = 0; i < N; i++) {
        float err = std::fabs(c_gpu[i] - c_cpu[i]);
        if (err > maxAbsErr) {
            maxAbsErr = err;
            worstIdx = i;
        }
    }

    cout << "max error |GPU-CPU| = " << maxAbsErr << " at i=" << worstIdx << endl;
    cout << "first 20 GPU results:" << endl;
    for (int i = 0; i < 20; i++) {
        cout << c_gpu[i] << " ";
    }
    cout << endl;

    if (maxAbsErr < 1e-6f)
        cout << "GPU calculations are correct!" << endl;
    else
        cout << "Mismatch detected!" << endl;

    return 0;
}
```

Файл lab08_1.cu

```
#include <cuda.h>
#include <cuda_runtime.h>
#include <cstdio>
#include <cstdlib>

__global__ void VecAddKernel(const float* a, const float* b, float* c, int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) {
        c[idx] = a[idx] + b[idx];
    }
}

static void checkCuda(cudaError_t e) {
    if (e != cudaSuccess) {
        std::printf("CUDA error: %s\n", cudaGetErrorString(e));
        std::exit(1);
    }
}

extern "C" void vec_add_cuda(const float *a, const float *b, float *c, int n) {
```

```
    int sizeBytes = n * (int) sizeof(float);

    float *a_gpu = nullptr, *b_gpu = nullptr, *c_gpu = nullptr;

    checkCuda(cudaMalloc((void**)&a_gpu, sizeBytes));
    checkCuda(cudaMalloc((void**)&b_gpu, sizeBytes));
    checkCuda(cudaMalloc((void**)&c_gpu, sizeBytes));

    checkCuda(cudaMemcpy(a_gpu, a, sizeBytes, cudaMemcpyHostToDevice));
    checkCuda(cudaMemcpy(b_gpu, b, sizeBytes, cudaMemcpyHostToDevice));

    dim3 threads(512, 1, 1);
    dim3 blocks((n + threads.x - 1) / threads.x, 1, 1);

    VecAddKernel<<<blocks, threads>>>(a_gpu, b_gpu, c_gpu, n);
    checkCuda(cudaGetLastError());
    checkCuda(cudaDeviceSynchronize());

    checkCuda(cudaMemcpy(c, c_gpu, sizeBytes, cudaMemcpyDeviceToHost));

    cudaFree(a_gpu);
    cudaFree(b_gpu);
    cudaFree(c_gpu);
}
```

Результаты

Сборка и выполнение тестовой программы (первая часть задания):

Сборка и выполнение программы (вторая часть задания):

Выводы

По результатам сравнения расхождения между значениями, полученными на GPU и CPU, не обнаружены. Следовательно, разработанное CUDA-ядро выполняет поэлементное сложение корректно, а перенос вычислений на GPU не приводит к ошибкам в результате.

Для компиляции CUDA-кода на Fedora 41 был установлен CUDA Toolkit (включающий компилятор `nvcc`) из официального репозитория NVIDIA. После установки `nvcc` находился в каталоге `/usr/local/cuda-13.2/bin`, поэтому для корректной работы командной строки путь к нему был добавлен в переменную окружения `PATH` (и при необходимости создан символьный линк `/usr/local/cuda` на установленную версию)

Это обеспечило возможность сборки проекта командой `nvcc <file>.cu <file>.cpp -o <app>`